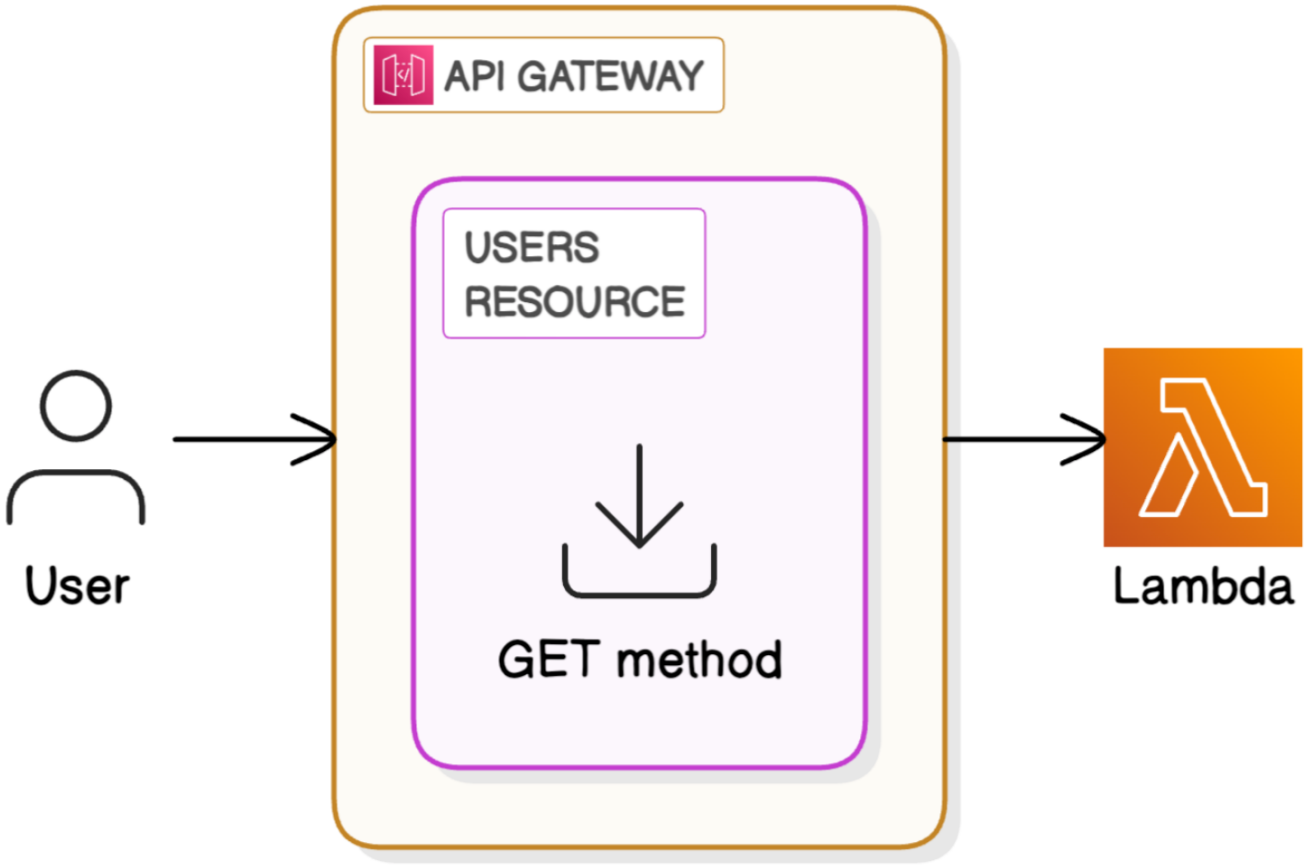
# APIs with Lambda + API Gateway - Step-by-Step Documentation




This documentation provides a comprehensive, step-by-step process for building a serverless API using AWS Lambda and Amazon API Gateway. It demonstrates how to configure Lambda functions, connect them to API Gateway, and deploy an API endpoint. This project forms the logic tier of a three-tier architecture solution.

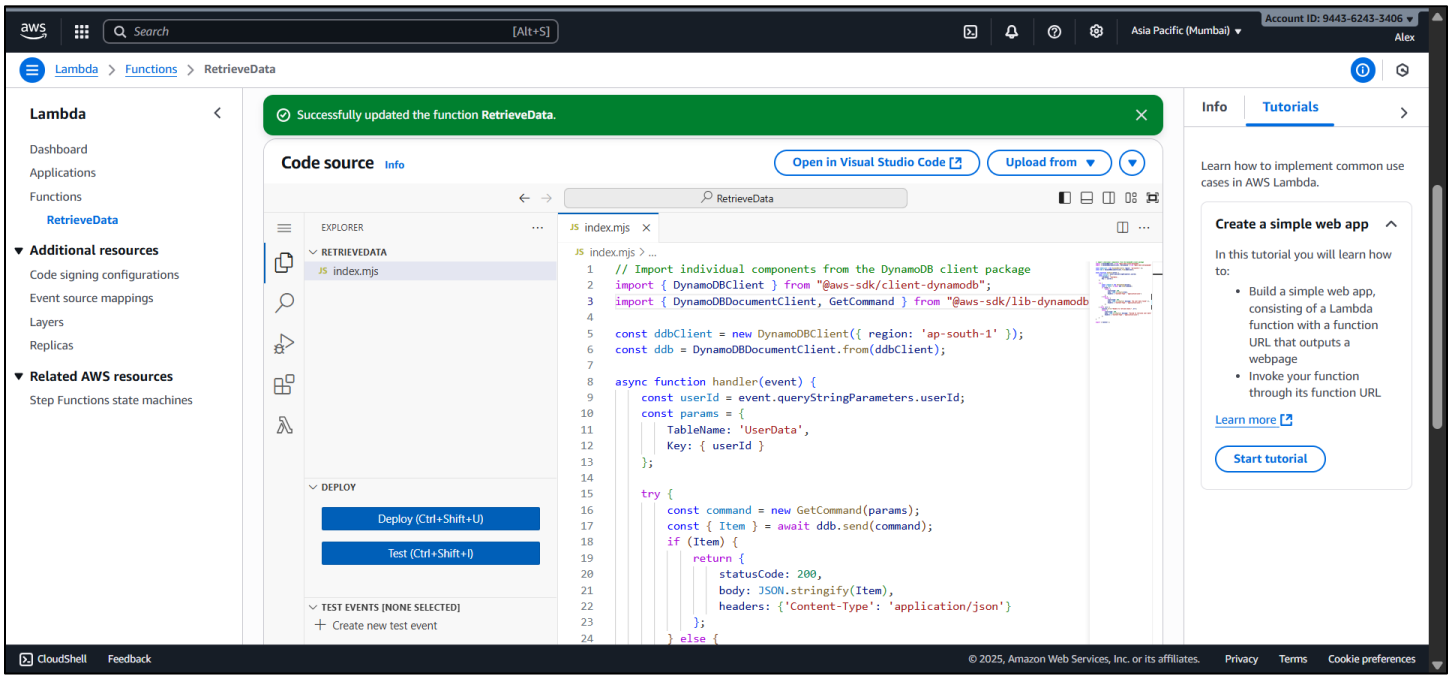
### Step 1: Create a Lambda Function

Set up a serverless function that will handle backend logic.

Actions:

- Log in to AWS Management Console.
- Navigate to the Lambda service.
- Click 'Create function'.
- Choose 'Author from scratch'.
- Enter Function name (e.g., RetrieveUserData).
- Select Runtime = Node.js and Architecture = x86_64.
- Click 'Create function'.

- Add the provided JavaScript code in the code editor.
- Click 'Deploy' to save the code.

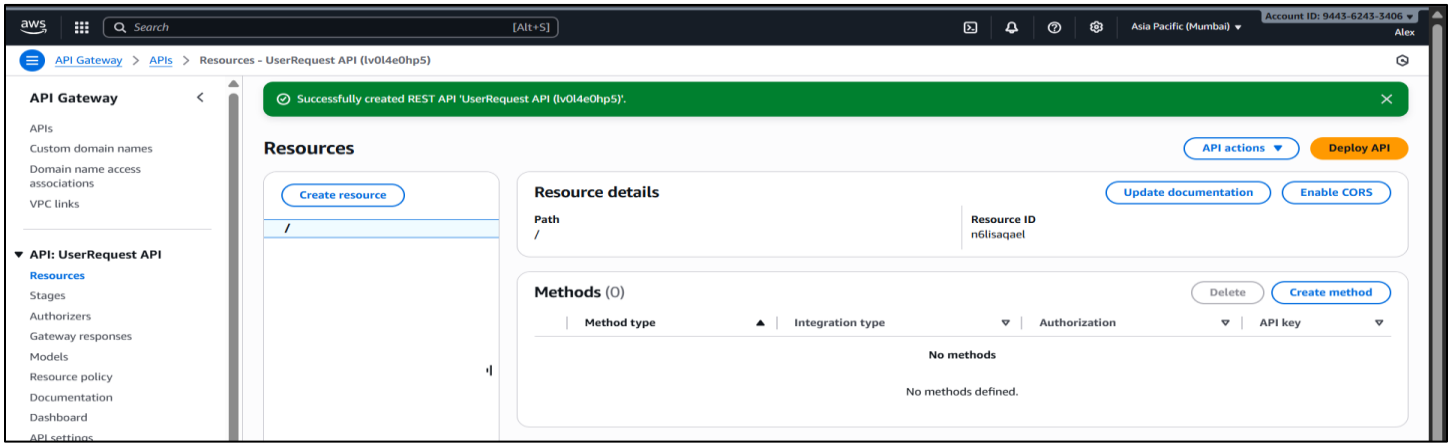



## Step 2: Set up API Gateway

Create an API that will connect users to the Lambda function.

Actions:

- Navigate to API Gateway in the AWS Console.
- Select 'Create API'.
- Choose 'REST API' and select 'Build'.
- Select 'New API' and provide API name (e.g., UserRequestAPI).
- Choose Endpoint type = Regional.
- Click 'Create API'.

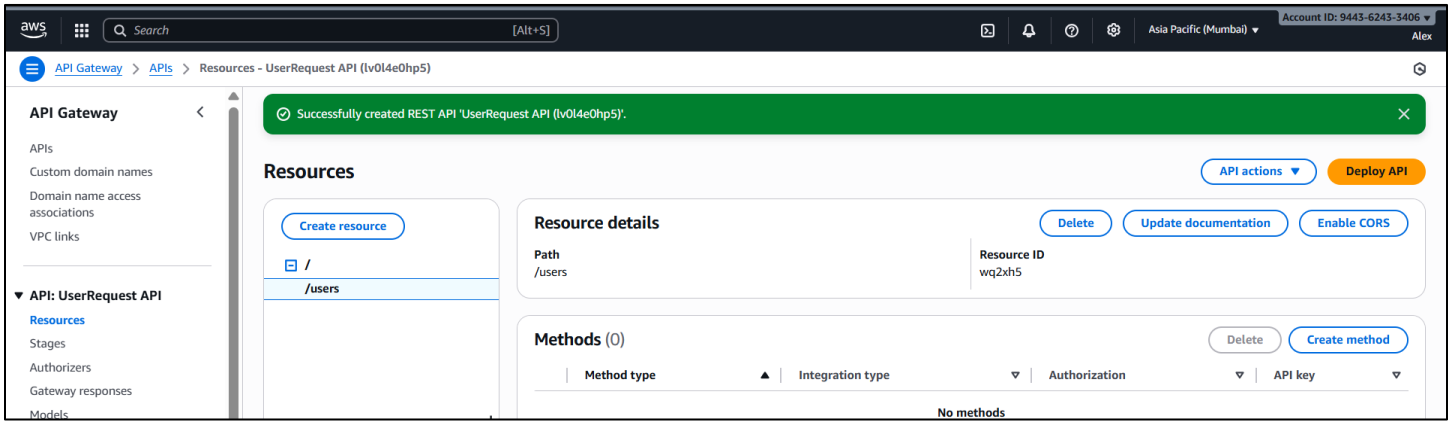


## Step 3: Create an API Resource

Define a resource path for your API.

Actions:

- In the API Gateway console, under Resources, select 'Create resource'.
- Enter Resource name = users.
- Confirm creation of the resource.

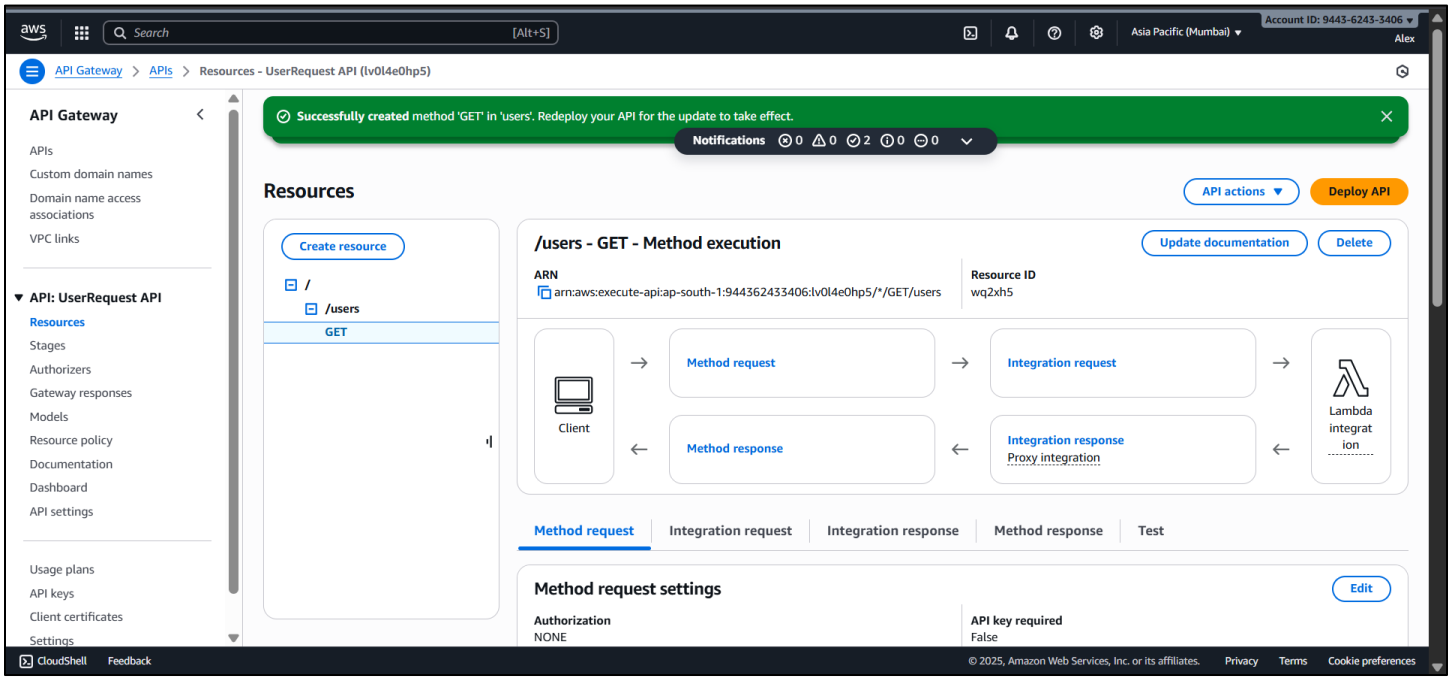



## Step 4: Create an API Method

Add a GET method to your resource and link it to Lambda.

Actions:

- Under the /users resource, click 'Create method'.
- Select GET.
- Choose Integration type = Lambda Function.
- Enable 'Lambda Proxy Integration'.
- Enter the name of your Lambda function (RetrieveUserData).
- Save the method configuration.

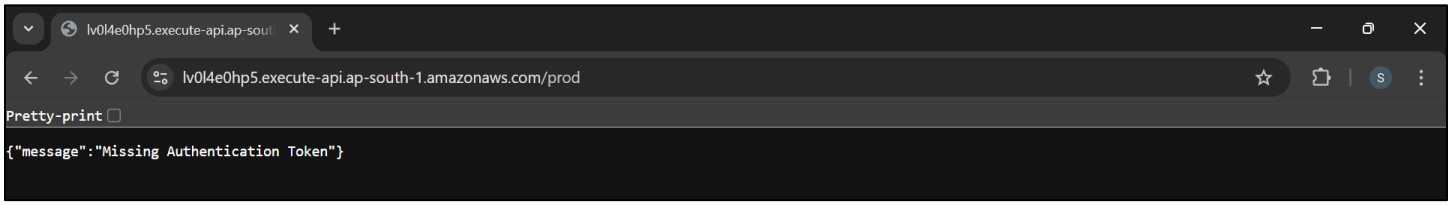



## Step 5: Deploy the API

Deploy the API to make it accessible via a public URL.

Actions:

- In API Gateway, select 'Deploy API'.
- Choose 'New stage'.
- Enter Stage name = prod.
- Click 'Deploy'.
- Copy the Invoke URL and test it in a browser.




Dang it - you'll get an error because we haven't set up our DynamoDB table yet. That's okay!